```
            if (ranks.contains(index + 1)) {
              Iterator(colValue)
            } else {
              Iterator.empty
            }
      })
    }).collectAsMap()
  }

  def findRankStatistics(
    pairRDD: RDD[(Int, Double)],
    ranks: List[Long]): Map[Int, Iterable[Double]] = {
    assert(ranks.forall(_ > 0))
    pairRDD.groupByKey().mapValues(iter => {
      val sortedIter  = iter.toArray.sorted
      sortedIter.zipWithIndex.flatMap(
        {
        case (colValue, index) =>
            if (ranks.contains(index + 1)) {
              //this is one of the desired rank statistics
              Iterator(colValue)
            } else {
              Iterator.empty
            }
        }
      ).toIterable //convert to more generic iterable type to
match out spec
    }).collectAsMap()
  }
```

This solution has several advantages. First, it gives the correct answer. Second, it is very short and easy to understand. It leverages out-of-the-box Spark and Scala functions and so it introduces few edge cases and is relatively easy to test. On small data, particularly if the input data has many columns but few records, it is actually relatively efficient because it only requires one shuffle in the `groupByKey` step and because the sorting step can be computed as a narrow transformation on the executors.

> **NOTE**
>
> In this function, we do use `collectAsMap`, which can have the same issues as `collect` that we warned you about earlier. In this instance, however, the danger of

memory errors is minimal because at the point of collecting, we know the number of keys and the number of values per each key. The number of keys is exactly the number of columns, which we have assumed to be no larger than a few thousand. The number of values is equal to the length of the rank statistics list we used as input for the function, which was not originally stored in a distributed way. However, it might be good practice to add a limit to the size of the input list to prevent failures in the collect step.

In the environment that we were using and on data with 10,000 rows and a few thousand columns, this solution was orders of magnitude faster than the one presented in "The Goldilocks Example" in which we looped through the columns iteratively and sorted each one. However, on data with a few million rows, we found that the solution failed consistently with memory exceptions even on a many-node cluster.

WHY GROUPBYKEY FAILS

If you have read *Learning Spark* or spent much time working with Spark at scale, the results of the `groupByKey` approach to solving the Goldilocks problem shouldn't surprise you as `groupByKey` is known to cause memory errors at scale. The reason is that the "groups" created by `groupByKey` are always iterators, which can't be distributed. This causes an expensive "shuffled read" step in which Spark has to read all of the shuffled data from disk and into memory.

Figure 6-1 is a screenshot taken from the Spark web UI that illustrates the high cost of `groupByKey`.

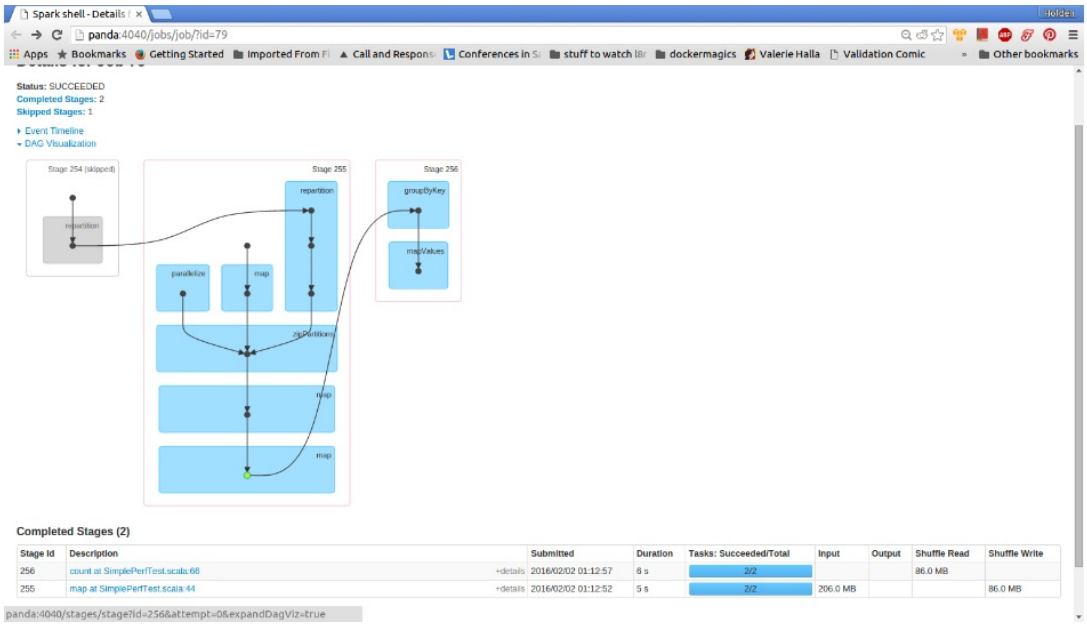

Figure 6-1. GroupByKey DAG and shuffled read

Notice that in this computation, the shuffled read is 86 MB even though the input data is about 200 MB. In other words, Spark has to read almost all of the shuffled data into memory.

As a consequence of partitioning by the hash value of the keys and pulling the result into memory to group as iterators, `groupByKey` often leads to out-of-memory errors on the executors if there are many duplicate records per key. Each record whose key has the same hash value must live in memory on a single machine. Thus, if just one of your keys contains too many records to fit in memory on one executor, the entire operation will fail.

Figure 6-2 illustrates a `groupByKey` operation of data about handlebar mustaches. As you can see, there are more records corresponding to the 94110 zip code, which is the zip code for the Mission District in San Francisco. Even if the records fit on the executors when evenly distributed, all the records associated with the 94110 zip code will not fit on a single executor after the `groupByKey` step.

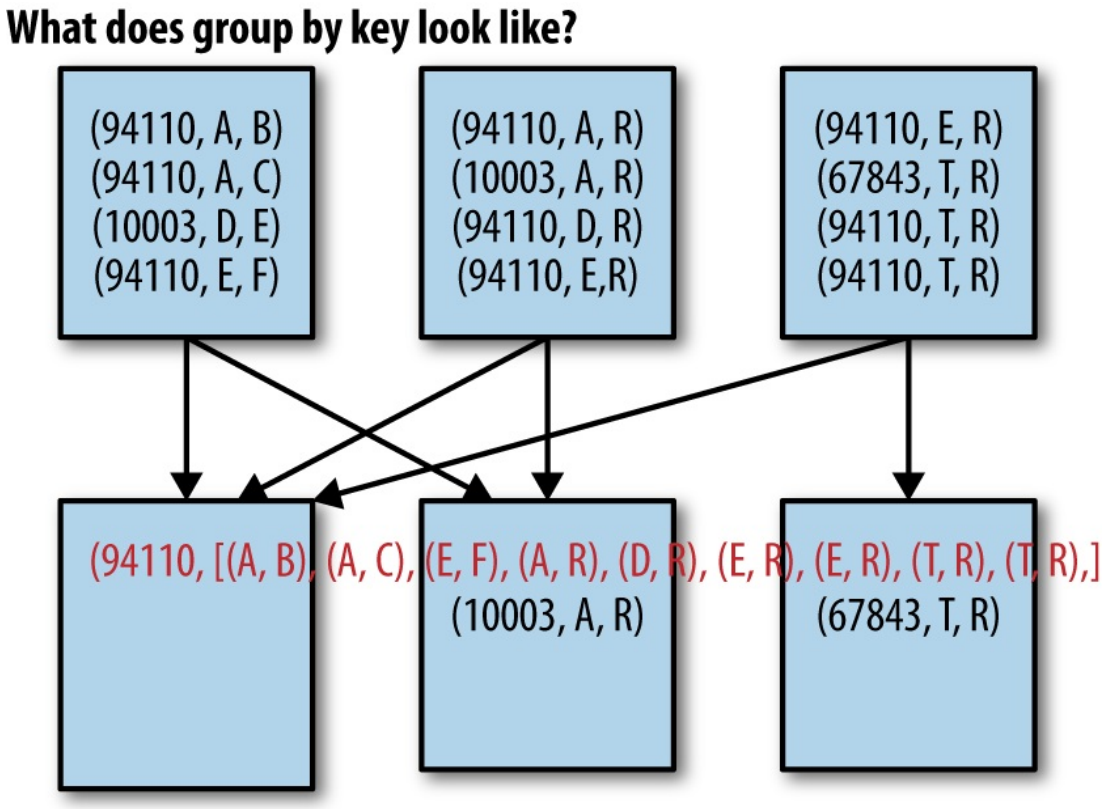

Figure 6-2. GroupByKey tip over

In general it is better to choose aggregation operations that can do some map-side reduction to decrease the number of records by key before shuffling (e.g., `aggregateByKey` or `reduceByKey`). If this is not possible, using a wide transformation that does not require all the values associated with one key to be kept in-memory as we discuss in "Secondary Sort and repartitionAndSortWithinPartitions" is a good alternative to `groupByKey`. If you must use `groupByKey`, it is best if the next operation is an iterator-to-iterator transformation as discussed in "Iterator-to-Iterator Transformations with mapPartitions".

Choosing an Aggregation Operation

Shuffling the records to combine those with the same key is a common use case for key/value Spark operations, and Spark provides a number of such

aggregation operations. Most of them are built atop the generic `combineByKey` operation, but they differ widely in performance. In this section we will detail these operations and some of the performance considerations associated with them.

Dictionary of Aggregation Operations with Performance Considerations

Aggregation operations have specific performance considerations, which we summarize in Table 6-3.

Table 6-3. A dictionary of Spark's key/value aggregation operations

Function	Purpose	Key restriction	Runs out of memory when	Slow
`groupByKey`	Group values with the same key into a single iterator.	Cannot have array keys with the default `HashPartitioner`. To use array keys, use a custom partitioner.	If all of the records associated with any single key take up too much space in-memory to be read from disk on one executor.	If the know this c The s more the nu distin increa numb per k the re evenl acros
`combineByKey`	Combine values with the same key using a different result type.	Same as above.	The "combine by" routine uses too much memory or creates too much garbage collection overhead, or the accumulator for one key becomes too large (this is the problem in `groupByKey`).	Same *can* b `grou` comb a redu

`aggregateByKey`	Same as `combineByKey`, but uses one zero value for all accumulators.	Same as above.	Same as above, but if implemented well less likely to cause garbage collection errors (see "Minimizing Object Creation").	See a gener `comb` since the m side b to a c
`reduceByKey`	Combine values with the same key. Reduction must be to same type as original values	See above, but often less expensive than `combineByKey` since `aggregateByKey` supports reusing the accumulator object to avoid object creation.	See above, but the type restriction makes memory errors unlikely. So long as the combine is not to a collection type, the function is probably reducing. Garbage collection is less than `aggregateByKey` since no additional accumulator object is created.	Same `aggr`
`foldByKey`	Combine values with the same key using an associative combine function and a zero value, which can be added to the result an arbitrary number of times. Use instead of `reduceByKey` when a natural `0` exists.	See above.	See above.	See a Perfo nearly `redu`

> **TIP**
>
> To avoid memory allocation in `aggregateByKey`, modify the accumulator rather than return a new one. See "Minimizing Object Creation".

PREVENTING OUT-OF-MEMORY ERRORS WITH

AGGREGATION OPERATIONS

`CombineByKey` and all of the aggregation operators built on top of it (`reduceByKey`, `foldLeft`, `foldRight`, `aggregateByKey`) are no better than `groupByKey` in terms of memory errors *if* they cause the accumulator to become too large for one key. In fact, if you look up the implementation of `groupByKey`, you can see that it is actually implemented using `combineByKey` where the accumulator is an iterator with all the data. Thus, the accumulator is the size of all the data for that key. In other words, these operations are unlikely to cause memory errors as long as the combining steps make the data smaller. However, if the accumulator gets larger with the addition of each new record, it will eventually cause memory errors if there are many records associated with one key.

Imagine doing a back-of-the-envelope memory calculation on your sequence and combine operators:

Given the sequence operation:

```
SeqOp(acc, v) => acc'
```

…and the combine operation:

```
combOp(acc1, acc2) = acc3.
```

Calculate whether:

```
memory(acc') < memory(acc) + memory(v)
and memory(acc3) < memory(acc1) + memory(acc2).
```

If so, the function is likely a reduction. If not, as is the case in `groupByKey`, you may need to consider a different strategy.

Beyond being less likely to run out of memory than `groupByKey`, the following four functions—`reduceByKey`, `treeAggregate`, `aggregateByKey`, and `foldByKey`—are implemented to use map-side combinations, meaning that records with the same key are combined before they are shuffled. This can greatly reduce the shuffled read. Compare the shuffled read in our original `groupByKey` (Figure 6-1) to the amount in `reduceByKey` in Figure 6-3. Recall that in the `groupByKey` case, our shuffled read was close to the size input. However, applying `reduceByKey` to the same input data reduces that number to a few hundred kilobytes!

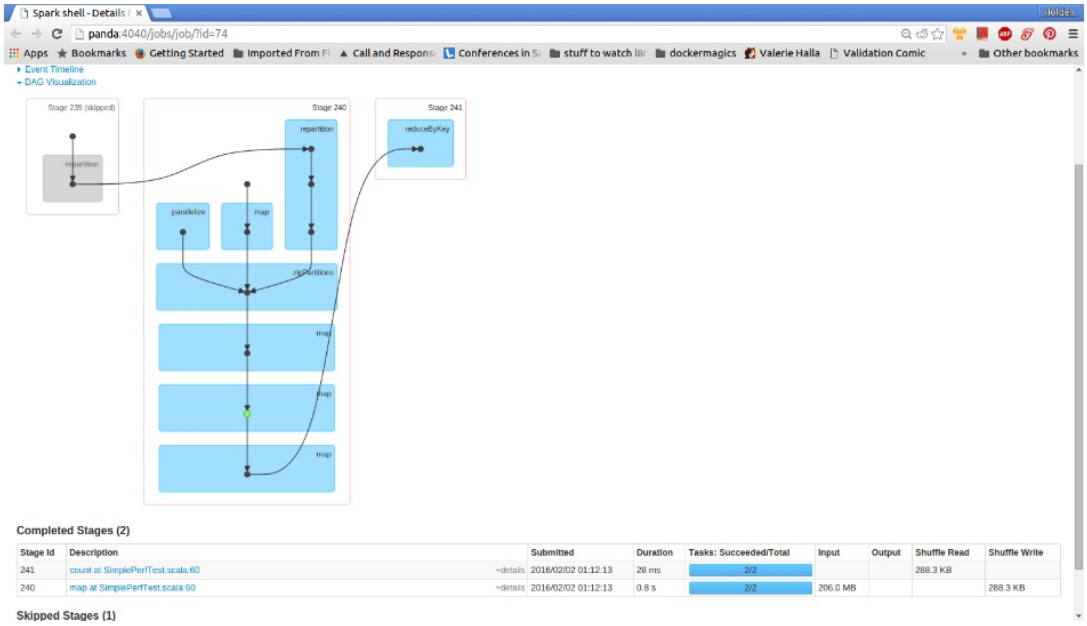

Figure 6-3. ReduceByKey DAG and shuffled read

Multiple RDD Operations

Some transformations can operate on multiple RDD inputs. The most

obvious of these are join type operations, but they are far from the only ones.

Co-Grouping

Much in the same way all of the accumulator operations (`reduceByKey`, `aggregateByKey`, `foldByKey`) are implemented using `combineByKey`, all of the join operations are implemented using the `cogroup` function, which uses the `CoGroupedRDD` type. A `CoGroupedRDD` is created from a sequence of key/value RDDs, each with the same key type. `cogroup` shuffles each of the RDDs so that the items with the same value from each of the RDDs will end up on the same partition and into a single RDD by key.

The `PairRDDFunctions` class provides several signatures for `cogroup`. `cogroup` and its alias `groupWith` can take one, two, or three RDDs, with the same key type as arguments (regardless of value type), return an RDD with each key, and then return a tuple of `Iterable` objects where each `Iterable` is all the values of the RDDs for that key.

Suppose, for example, that we had two datasets of information about each panda: one with the scores in a series of games, and one with their favorite foods. We could use `cogroup` to associate each panda's ID with an iterator of their scores and another iterator of their favorite foods, as in Example 6-5.

Example 6-5. Cogroup example

```
   val cogroupedRDD: RDD[(Long, (Iterable[Double],
Iterable[String]))] =
     scoreRDD.cogroup(foodRDD)
```

`cogroup` can be useful as an alternative to join when joining with multiple RDDs. Rather than doing joins on multiple RDDs with one RDD it is more performant to co-partition the RDDs since that will prevent Spark from shuffling the RDD being repeatedly joined.

For example, if we needed to join the panda score data with both address and favorite foods, it would be better to use `cogroup` than two join operations, as shown in Example 6-6.

Example 6-6. Cogroup to avoid multiple joins on the same RDD

```
val addressScoreFood = addressRDD.cogroup(scoreRDD, foodRDD)
```

Despite its advantages, `cogroup` will cause memory errors for the same reason as `groupByKeys` if one key in either RDD or both combined is associated with more data than will fit on a single partition. In particular, `cogroup` requires that all the records in all of the co-grouped RDDs for one key be able to fit on one partition. For more information on joins, see Chapter 4.

Partitioners and Key/Value Data

As we covered in Chapter 2, a partition in Spark represents a unit of parallel execution that corresponds to one task. An RDD without a known partitioner will assign data to partitions according only to the data size and partition size.[3] The partitioner object defines a mapping from the records in an RDD to a partition index. By assigning a partitioner to an RDD, we can guarantee something about the records on each partition—for example, that it falls within a given range (range partitioner) or includes only elements whose keys have the same hash code (hash partitioner).

There are three methods that exist exclusively to change the way an RDD is partitioned. For RDDs of a generic record type, `repartition` and `coalesce` can be used to simply change the number of partitions that the RDD uses, irrespective of the value of the records in the RDD. As we discussed in "The Special Case of coalesce", `repartition` shuffles the RDD with a hash partitioner and the given number of partitions. (We will explain in more detail how hash partitioning works in "Hash Partitioning".) `coalesce`, on the other hand, is an optimized version of `repartition` that avoids a full shuffle if the desired number of partitions is less than the current number of partitions. Recall that when `coalesce` reduces the number of partitions, it does so by merely combining partitions—and thus `coalesce` is not a wide transformation since the partition can be determined at design time. When `coalesce` increases the number of partitions it has the same behavior as `repartition`. For RDDs of key/value pairs, we can use a function called `partitionBy`, which takes a partition object rather than a number of partitions and shuffles the RDD with the new partitioner. `PartitionBy` allows for much more control in the way that the records are partitioned since the partitioner supports defining a function that assigns a partition to a record based on the value of that key.

In all cases, `repartition` and `coalesce` do not assign a known partitioner to the RDD. In contrast, using `partitionBy` (and most other key/value functions that cause a shuffle) results in an RDD with a known partitioner. In some instances, when an RDD has a known partitioner, Spark can rely on the information about data locality provided by the partitioner to avoid doing a shuffle even if the transformation has wide dependencies. The rest of the sections in this chapter will be about ways to leverage knowledge about partitioning or use custom partitioning to

“shuffle less” and “shuffle better.” Specifically, we aim to use this information to minimize the number of times a program causes a shuffle, the distance the data has to travel in a shuffle, and the likelihood that a disproportionate amount of the data will be sent to one partition, thus causing out-of-memory errors or straggler tasks.

Using the Spark Partitioner Object

Conceptually, the partitioner defines how records will be distributed and thus which records will be completed by each task. Practically, a partitioner is actually an interface with two methods—`numPartitions` and `getPartition`. `numPartitions` defines the number of partitions in the RDD after partitioning. `getPartition` defines a mapping from a key to the integer index of the partition where records with that key should be sent. There are two implementations for the partitioner object provided by Spark: the `HashPartitioner` and `RangePartitioner`. If neither of these suit your needs, it is possible to define a custom partitioner.

Hash Partitioning

The default partitioner for pair RDD operations (not ordered RDD operations) is a `HashPartitioner`. A `HashPartitioner` determines the index of the child partition based on the hash value of the key. The hash partitioner requires a `partitions` parameter, which determines the number of partitions in the output RDD and the number of bins used in the hashing function. If unspecified, Spark uses the value of the `spark.default.parallelism` value in the `SparkConf` to determine the number of partitions. If the default parallelism value is unset, Spark defaults to the largest number of partitions that the RDD has